SAE3.01 COURSE D'ORIENTATION V3.0
TECHNOLOGIE LORAWAN

INFORMATIONS

Application TTN	GPS Lora
Région TTN	Europe
DevEUI	8CF9572000065F6A
JoinEUI	8CF9572000000000
AppKey	3202E94C9CEF71387D5B2F208A12C683
EndDeviceID	gpsbadolleraquin

SOMMAIRE

1. INTRODUCTION ET OBJECTIFS

2. CAHIER DES CHARGES

3. PHASE 1 : PROTOTYPAGE SUR THE THINGS NETWORK (TTN)

4. PHASE 2 : MIGRATION VERS INFRASTRUCTURE PRIVÉE

5. CONCLUSION ET DIFFICULTÉS

1. INTRODUCTION ET OBJECTIFS

L'objectif de cette SAE est de concevoir une infrastructure de suivi pour une course d'orientation autour de Roanne.

- **Le défi :** Suivre des balises (Trackers) en temps réel dans une forêt.
- **La stratégie :** Nous avons découpé le projet en deux phases distinctes :
    1. **Phase de validation (POC) :** Utilisation de The Things Network (TTN) et Datacake pour valider le fonctionnement du code Arduino et du GPS.
    2. **Phase de production :** Déploiement d'une infrastructure privée sur Digital Ocean avec ChirpStack et développement d'une application Android.

Badolle, Raquin

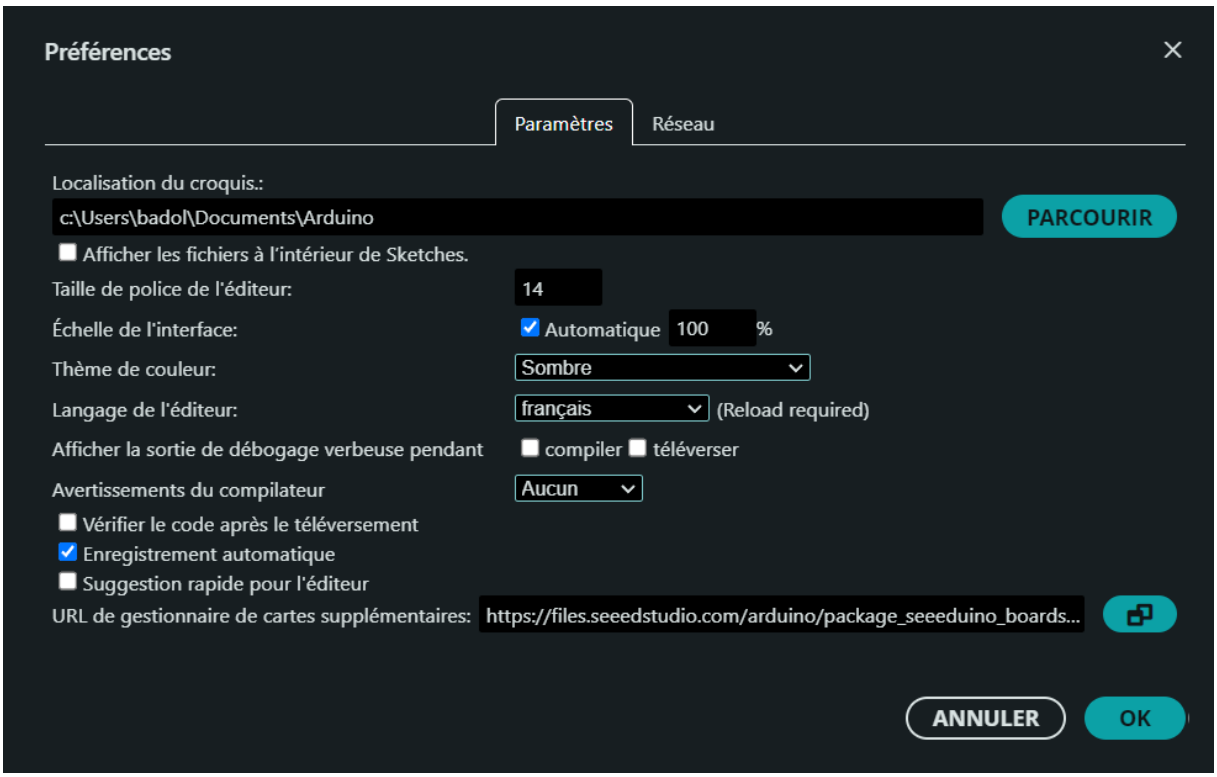
SAE3.01 COURSE D'ORIENTATION V3.0
TECHNOLOGIE LORAWAN

2. CAHIER DES CHARGES

L'objectif est de déployer une infrastructure de suivi pour une course d'orientation.

- Balises : Objets connectés (Trackers) cachés ou mobiles.
- Contrainte critique : Remontée de position GPS toutes les 10 secondes.
- Infrastructure : Doit évoluer d'une solution publique (TTN) vers une solution privée maîtrisée (ChirpStack sur Digital Ocean).
- Visualisation : Serveur Web pour les joueurs.

3. INSTALLATION ENVIRONNEMENT DE DEVELOPPEMENT



3.1 PROTOTYPAGE SUR THE THINGS NETWORK (TTN)

Avant de monter notre propre serveur, nous devions nous assurer que la carte SeeedStudio était capable d'envoyer une position GPS fiable.

SAE3.01 COURSE D'ORIENTATION V3.0 TECHNOLOGIE LORAWAN

2.1 Configuration Matérielle et Correctifs

Nous avons rencontré des problèmes de stabilité avec la configuration par défaut du wiki SeeedStudio. Nous avons dû appliquer un correctif matériel dans le code :

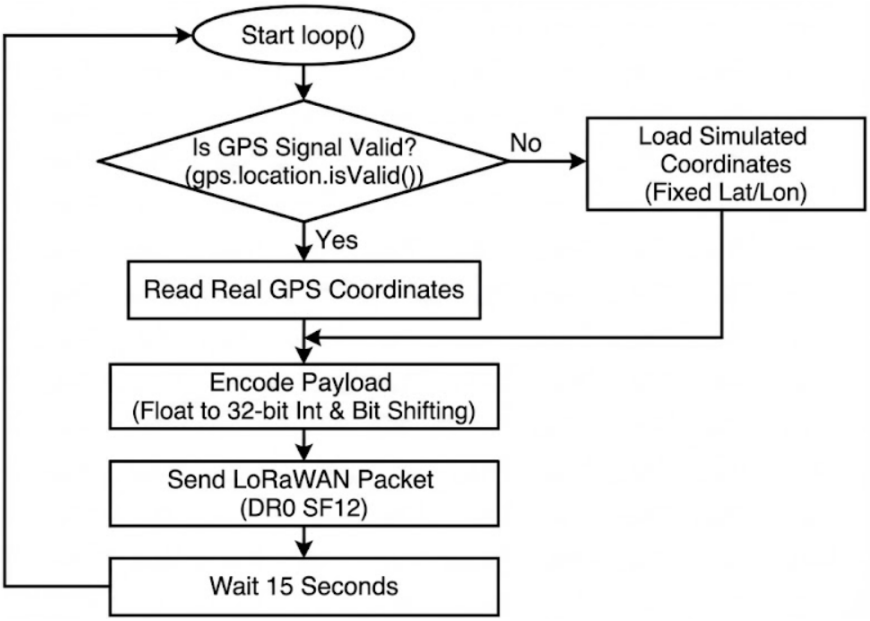
- **Problème :** L'activation de la Pin 50 faisait planter la carte.
- **Solution :** Nous pilotons uniquement la Pin 38 pour l'alimentation du GPS et nous utilisons Serial2 pour la communication.

2.2 Le Code Embarqué (Arduino)

Voici le programme développé et validé pour cette phase. Il gère la connexion OTAA, le "Fix" GPS, et l'envoi optimisé des données.

Configuration LoRaWAN :

- Device EUI : 8CF9572000065F6A
- App EUI : 8CF9572000000000
- Mode : DR0 (Data Rate 0) pour maximiser la portée en forêt ("Mode Forêt").



2.3 Visualisation sur Datacake

Une fois les données reçues sur TTN, nous avons configuré un Webhook vers Datacake.

- **Payload Decoder :** Sur TTN/Datacake, nous avons dû écrire un petit script JS pour décoder les 8 octets reçus et retrouver les coordonnées (diviser par 1 000 000).

SAE3.01 COURSE D'ORIENTATION V3.0
TECHNOLOGIE LORAWAN

Décodeur de Payload

Réglage pour toute la gamme de produits

Lorsque vos appareils envoient des données, le payload est transmis au décodeur payload, en même temps que le nom de l'événement. Le décodeur payload transforme ensuite les données en valeurs mesurées.

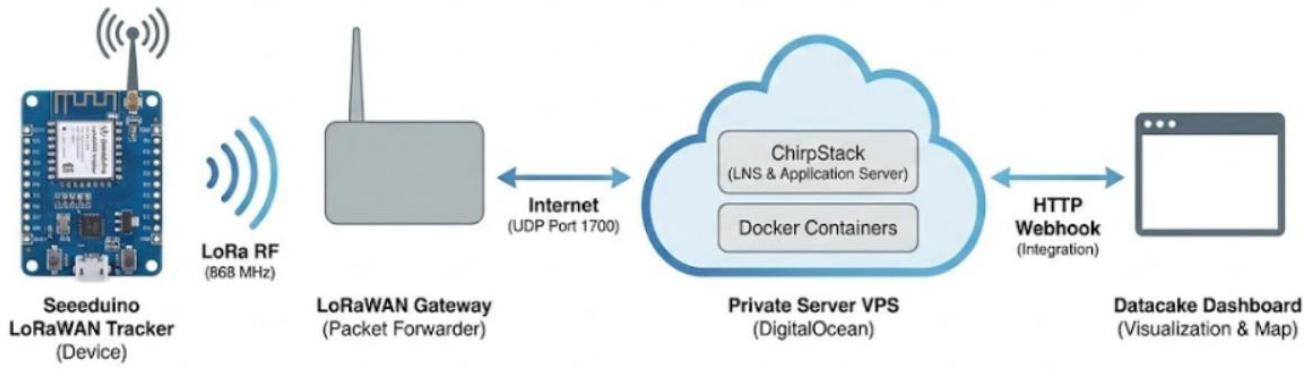
```
function Decoder(payload, metadata) {

    // CAS 1 : Si TTN a déjà fait le travail (ce qu'on a vu sur votre image)
    if (payload.latitude && payload.longitude) {
        return [
            { field: "latitude", value: payload.latitude },
            { field: "longitude", value: payload.longitude },
            // On crée le champ spécial "LOCATION" pour la carte
            { field: "location", value: "(" + payload.latitude + "," + payload.longitude + ")" }
        ];
    }

    // CAS 2 : Si Datacake reçoit les octets bruts (votre méthode mathématique)
    // Utile si le lien TTN n'envoie pas le JSON décodé
    var bytes = payload; // Datacake passe parfois les bytes directement ici
    if (Array.isArray(bytes) && bytes.length >= 8) {
        var lat = (bytes[0] << 24 | bytes[1] << 16 | bytes[2] << 8 | bytes[3]);
        var lon = (bytes[4] << 24 | bytes[5] << 16 | bytes[6] << 8 | bytes[7]);

        var lat_final = lat / 1000000;
        var lon_final = lon / 1000000;

        return [
            { field: "latitude", value: lat_final },
            { field: "longitude", value: lon_final },
            { field: "location", value: "(" + lat_final + "," + lon_final + ")" }
        ];
    }

    return [];
}
```

- **Résultat :** Affichage de la position sur une carte (Map Widget) et validation de la précision du tracker en extérieur.

3. PHASE 2 : MIGRATION VERS INFRASTRUCTURE PRIVÉE

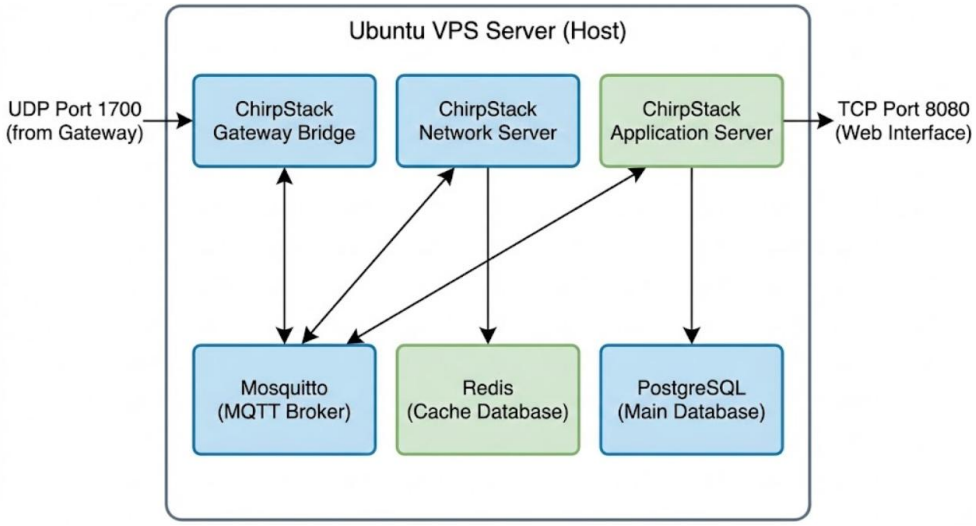
Une fois le code Arduino validé sur le réseau public (TTN), nous avons basculé vers une infrastructure que nous maîtrisons totalement.

SAE3.01 COURSE D'ORIENTATION V3.0
TECHNOLOGIE LORAWAN



3.1 Déploiement Serveur (Digital Ocean & Docker)

Nous avons loué un VPS chez Digital Ocean (Ubuntu Linux) et installé la stack **ChirpStack v4** via Docker Compose (incluant Mosquitto, PostgreSQL, Redis).



3.2 Reconfiguration de la Gateway

Nous avons reconfiguré la Dragino LPS8N pour pointer vers l'IP publique de notre serveur Digital Ocean via le port **1700 UDP**, abandonnant l'adresse eu1.cloud.thethings.network.

3.3 Configuration Avancée sur ChirpStack

ChirpStack agit comme le "cerveau" du réseau. Pour autoriser notre capteur à communiquer, nous avons suivi une procédure stricte :

1. **Création du Device Profile :** Nous avons défini un profil correspondant aux capacités techniques de la carte : protocole LoRaWAN 1.0.3, fonctionnement en Classe A, et région **EU868** (fréquence européenne).

2. **Enregistrement du Device :** L'appareil a été ajouté avec son **DevEUI** unique (8CF9572000065F6A) et configuré en mode **OTAA** (Over-The-Air Activation) pour une sécurité maximale.
3. **Gestion des Clés (Security) :** Nous avons saisi l'**AppKey** (3202E...) dans l'onglet "Keys". C'est cette clé qui assure le chiffrement des données entre le capteur et notre serveur privé.
4. **Validation de la Connexion :** Dans l'onglet "Events" de ChirpStack, nous avons pu valider le fonctionnement en observant la séquence d'authentification : un message JoinRequest émis par le capteur, immédiatement suivi d'un JoinAccept renvoyé par notre serveur.

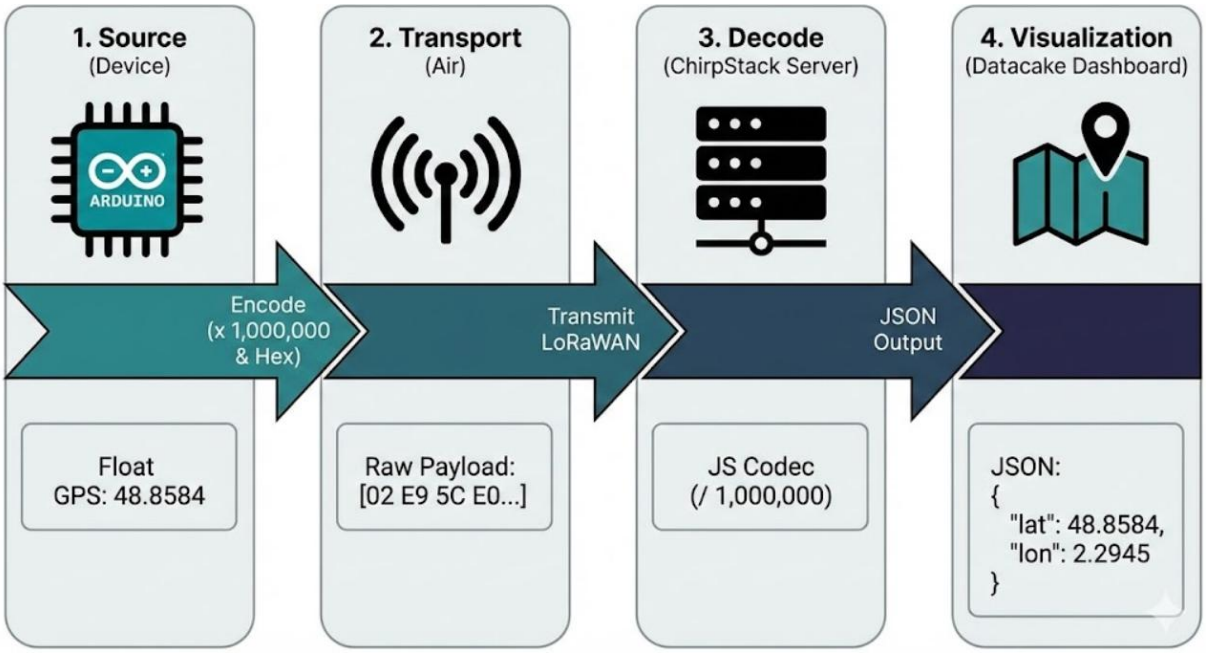
3.4 Pont ChirpStack Datacake (Webhook)

Pour visualiser les données reçues sur notre serveur privé sans développer immédiatement une application complexe, nous avons connecté ChirpStack à Datacake.

- **Intégration HTTP :** Dans ChirpStack (Menu Integrations > HTTP), nous avons configuré un **Webhook**.
- **Configuration :** Nous avons entré l'URL d'ingestion fournie par Datacake : https://api.datacake.co/integrations/lorawan/chirpstack/.
- **Fonctionnement :** Désormais, chaque trame LoRa reçue par notre serveur privé est automatiquement encapsulée dans un fichier JSON et "poussée" vers Datacake.

3.5 Visualisation et Décodage sur Datacake

SAE3.01 COURSE D'ORIENTATION V3.0
TECHNOLOGIE LORAWAN



Datacake reçoit les données brutes (octets) de notre serveur privé. Il a fallu les rendre lisibles pour l'utilisateur.

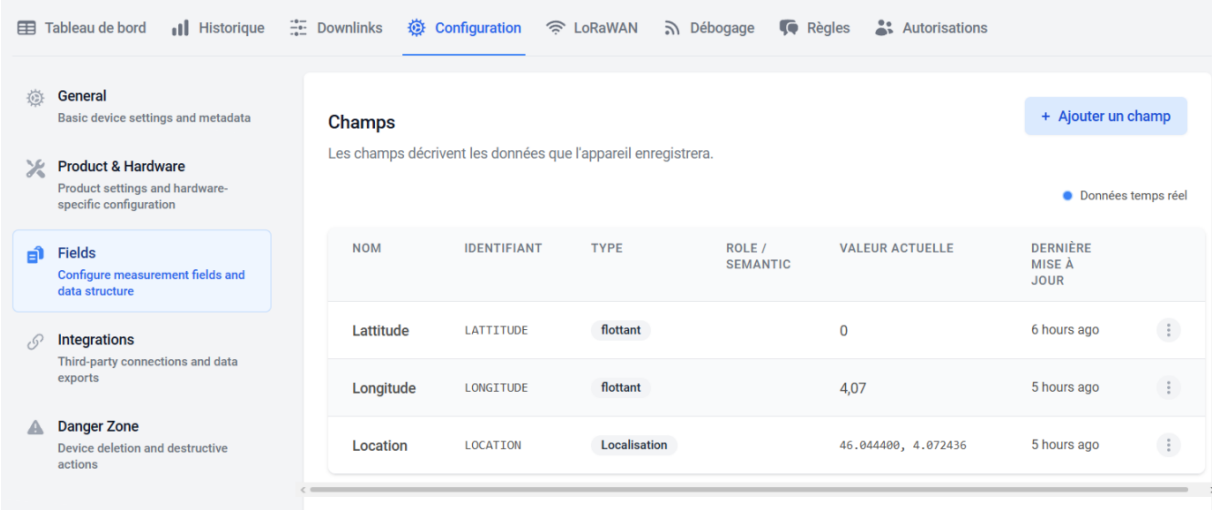
1. **Payload Decoder (JavaScript) :** Le capteur envoie la latitude et la longitude sous forme d'entiers pour économiser la bande passante. Nous avons intégré un script décodeur qui récupère les octets (ex: bytes[0]) et effectue l'opération inverse du microcontrôleur (division par 1 000 000) pour retrouver les coordonnées GPS (ex: 45.123456).

Codec functions

```
function decodeUplink(input) {
  // Décodage Latitude / Longitude (Big Endian)
  var decoded = {};

  if (input.bytes.length === 8) {
    var lat = (input.bytes[0] << 24 | input.bytes[1] << 16 | input.bytes[2] << 8 | input.bytes[3]) / 1000000;
    var lon = (input.bytes[4] << 24 | input.bytes[5] << 16 | input.bytes[6] << 8 | input.bytes[7]) / 1000000;
    decoded.latitude = lat;
    decoded.longitude = lon;
    decoded.status = "GPS Fix OK";
  } else {
    decoded.status = "Heartbeat (No Fix)";
  }

  return { data: decoded };
}
```

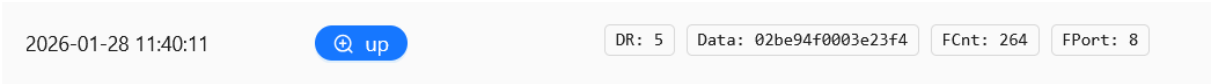
2. **Champs de Données (Fields) :** Nous avons structuré la base de données Datacake avec trois champs : Latitude (Float), Longitude (Float), et Location (Geopoint).

SAE3.01 COURSE D'ORIENTATION V3.0
TECHNOLOGIE LORAWAN



3. **Dashboard :** Un widget "Map" a été lié au champ Location. Cela nous a permis de valider en temps réel que notre infrastructure privée (Digital Ocean + ChirpStack) traitait correctement les positions GPS envoyées par la carte.

3.6 Test Réel



Badolle, Raquin

SAE3.01 COURSE D'ORIENTATION V3.0
TECHNOLOGIE LORAWAN

Details: 2026-01-28 11:40:11 Download

```
deviceProfileId: "8b099e7c-979c-4db8-bdbe-58df4b184ecf"
deviceProfileName: "Seeeduino-GPS"
deviceName: "Mon_Tracker"
devEui: "8cf9572000065f6a"
deviceClassEnabled: "CLASS_A"
tags: {} 0 keys
devAddr: "01343c27"
adr: true
dr: 5
fCnt: 264
fPort: 8
confirmed: false
data: "Ar6U8AA+I/Q="
object: {} 3 keys
  longitude: 4.072436
  latitude: 46.0444
  status: "GPS Fix OK"
rxInfo: [] 1 item
  0: {} 11 keys
    gatewayId: "a84041ffff253fa0"
    uplinkId: 42578
    gwTime: "2012-01-08T16:26:31.987749+00:00"
    nsTime: "2026-01-28T10:40:11.566786272+00:00"
    rssi: -60
    snr: 13.2
    channel: 1
    rfChain: 1
    location: {} 0 keys
    context: "Wlhd2Q=="
    crcStatus: "CRC_OK"
txInfo: {} 2 keys
  frequency: 868300000
  modulation: {} 1 key
    lora: {} 3 keys
```


Vidéo : https://mega.nz/file/vlAARLLQ#B3NhSvT5hHGkdywChia0ZRxw5B2-C_yoYaeeFbr4PQI

Nous pouvons constater que la réception de donnée et le déplacement de la localisation est bien fonctionnel

5. CONCLUSION ET DIFFICULTÉS

Ce projet a permis de mettre en place une chaîne IoT complète. Les principales difficultés surmontées furent le plantage matériel (résolu par correctif Pin 38) et la configuration fine des clés de sécurité sur le serveur privé ChirpStack. L'infrastructure est désormais opérationnelle pour la course.

Badolle, Raquin